Introduction to Programming

W07 In-Depth Object Orientation

Applied Education Technologies

26 November 2025
Technical University of Munich

Schedule

#	Date	Subject
1	15.10.25	Introduction
2	22.10.25	Control Flow in Programming
3	29.10.25	**In-Depth Core Concepts***
4	05.11.25	Core Data Structures
5	12.11.25	Code Reuse and Structure
6	19.11.25	Type Flexibility and Safety
7	**26.11.25**	**In-Depth Object Orientation***
8	03.12.25	Functional Programming Essentials
9	10.12.25	Algorithms and Data Handling
10	17.12.25	Programming Languages
11	07.01.26	Graphical User Interfaces
12	14.01.26	Recursion
13	21.01.26	Concurrency
14	28.01.26	Beyond Programming
15	04.02.26	Course Review

*** Repetition**

Roadmap

- **Context**
  - Apply the basics of object-oriented programming
  - Use basic control structures (**`if`**, **`switch`**, **`for`**, **`while`**)
  - Implement and use basic data types (**`List`**, **`Stack`**, **`Queue`**)
  - Apply abstraction, encapsulation, inheritance, and polymorphism
- **Learning goals**
  - Apply inheritance, abstract classes, and interfaces
  - Apply polymorphism
  - Apply generics
  - Understand the rules for the supervised exercise
  - Use the Artemis exam mode

Outline

- **Recap: Inheritance**
- Recap: Abstract classes, interfaces and polymorphism
- Recap: Generics
- Test supervised exercise

Recap: inheritance

- Mechanism where a class (child or subclass) can inherit attributes and methods from another class (parent or superclass)
- Promotes code reuse and logical hierarchy
- `extends` is used to define a subclass that inherits from a superclass

The parent class

```java
class Animal {
    String name;
    public Animal(String name) {
        this.name = name;
    }
    public void speak() {
        System.out.println(name + " makes a sound");
    }
}
```

Child class that inherits from the super class

```java
class Dog extends Animal {
    public Dog(String name) {
        super(name); // Call the parent class constructor
    }
}
```

The super keyword allows a subclass to refer to its immediate superclass, useful for calling parent class constructors or methods

Inheritance

- Access levels: inherited members follow access control rules (public, protected, private)
  - public and protected members of the superclass are accessible in the subclass
  - private members are not directly accessible but can be accessed indirectly via public/ protected methods in the superclass
- Method overriding: subclasses can override methods of the superclass to provide a specific implementation

```java
class Animal {
    String name;
    public Animal(String name) {
        this.name = name;
    }
    public void speak() {
        System.out.println(name + " makes a sound");
    }
}
```

```java
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }
    @Override
    public void speak() {
        System.out.println(name + " barks");
    }
}
```

Overrides the implementation found in the super class

Example

```java
class Animal {
    String name;
    public Animal(String name) {
        this.name = name;
    }
    public void speak() {
        System.out.println(name + " makes a sound");
    }
}
```

```java
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }
    @Override
    public void speak() {
        System.out.println(name + " barks");
    }
}
```

Overrides the implementation found in the super class

```java
public class Main {
    public static void main(String[] args) {
        Animal theAnimal = new Animal("The Animal");
        Dog dog = new Dog("Buddy");

        theAnimal.speak();     // Output: The Animal makes a sound
        dog.speak();           // Output: Buddy barks
    }
}
```

W07E02 Inheritance

Not started yet.

Start exercise

Easy

Due tonight

10 min

1. Define a class `Institution` with
   1. A `String` attribute called `name`
   2. A constructor to initialize `name`
   3. A method `describe()` that prints a description of the institution
2. Define a subclass `University`
   1. Add an attribute `numberOfStudents`
   2. Override the `describe()` method. Call `Institution`'s `describe()` method and print the number of students
3. In `main()`, create instances of `University` and call `describe()` on them

Outline

- Recap: Inheritance
- **Recap: Abstract classes, interfaces and polymorphism**
- Recap: Generics
- Test supervised exercise

Recap: abstract classes

- An abstract class contains abstract methods (methods without implementation), and serves as a blueprint for grouping subclasses that provide different implementations of those methods
- An abstract class cannot be instantiated

Defining a class as abstract

```
abstract class ChristmasDecoration {

    abstract String sparkle();

    void displayType() {
        System.out.println("This is a Christmas
        decoration.");
    }

}
```

Defining a method as abstract

Note that here we don't have a method body!

Extend the super class

Implement the method

```
class ChristmasTree extends ChristmasDecoration {
    private String treeType;
    ChristmasTree(String treeType) {
        this.treeType = treeType;
    }
    @Override
    String sparkle() {
        return "The " + treeType + " tree sparkles with
bright, colorful lights!";
    }
}
```

Recap: interfaces

- A blueprint that defines a set of methods a class must implement, but it does **not** provide the actual method implementations
- Allow different classes to share common behavior

Defining an interface

Methods do not have a body!

```
public interface Decorative {
        String illuminate();
        String decorate();
}
```

Keyword for interfaces

You have to implement all methods defined in the interface

```
class ChristmasTree extends ChristmasDecoration implements Decorative {
/* continuation of the class ChristmasTree */
    @Override
    public String illuminate() {
        return "The " + treeType + " tree glows with soft, golden light.";
    }
    @Override
    public String decorate() {
        return "The " + treeType + " tree is adorned with baubles, ribbons,
and a shining star.";
    }
}
```

Recap: polymorphism

- Allows objects of related classes to be treated as objects of a common superclass or interface, enabling methods to behave differently based on the object's actual class
- Can be achieved through
  - Method overriding (redefine a method in a subclass)
  - Method overloading (define multiple methods with the same name but different parameters)

```java
abstract class ChristmasDecoration {
    abstract String sparkle();

    // Normal method that can be overloaded
    void displayType() {
        System.out.println("This is a Christmas decoration.");
    }
    // Overloaded method (polymorphism)
    void displayType(String type) {
        System.out.println("This is a " + type + " decoration.");
    }
}
```

W07E03 Polymorphism

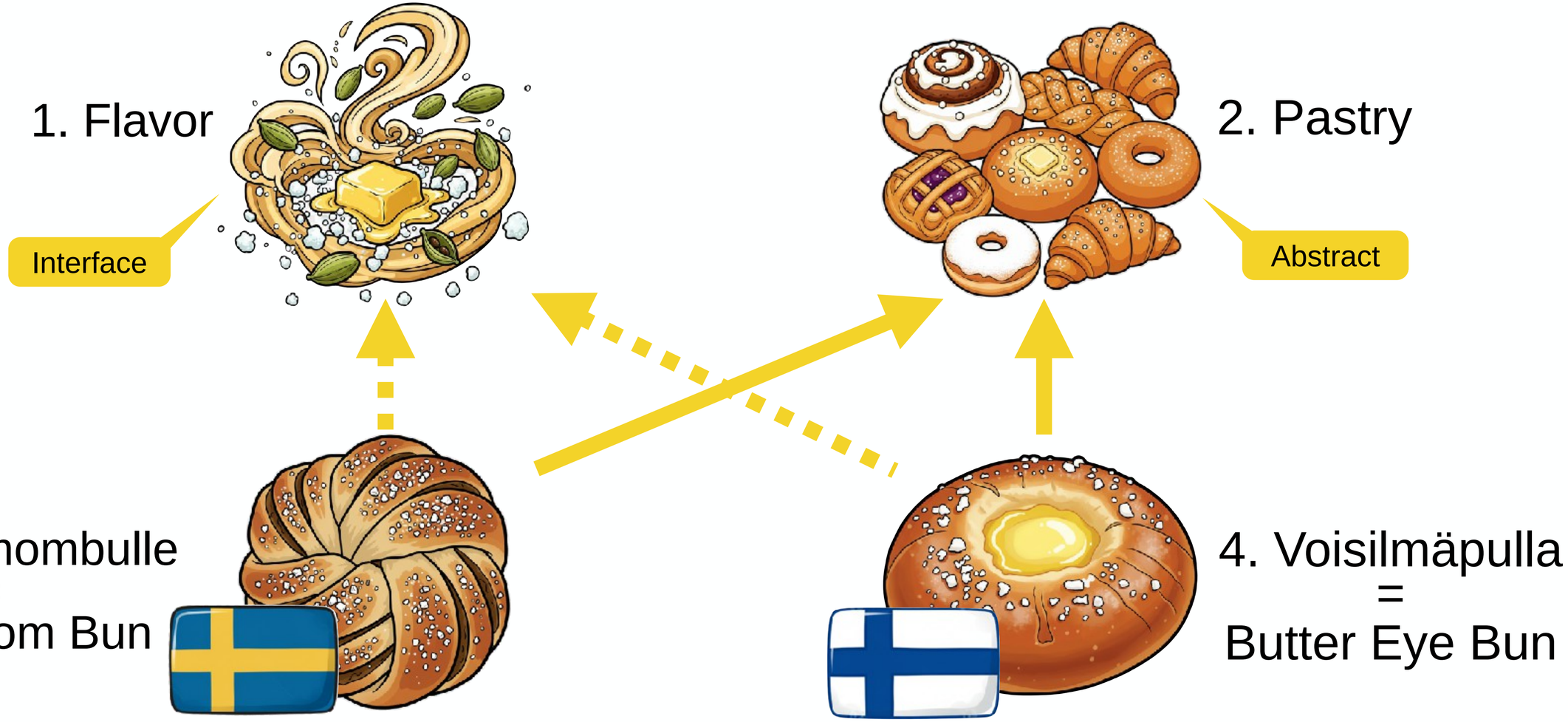
Start exercise

Easy

Not started yet.

Due tonight

10 min



Outline

- Recap: Inheritance
- Recap: Abstract classes, interfaces and polymorphism
- **Recap: Generics**
- Test supervised exercise

Recap: using generics

- You've already used them for `Collection`s
  - `List<String>`: a list of strings
  - `List<Train>`: a list of trains
- Dynamically impose which type is allowed in a data structure

Recap: generic classes

- Creating your own classes with generics
  - `public class Present<T> {`
  - `public class Present<Content> {`

Generic types are often just "T", "E", "C", or some other single chars

But it can also be any other (valid) name

- You can use it in this class like any other type
  - `private T content;`
  - `public T getContent() {...}`
  - `public void setContent(T content) {...}`

Recap: generic bounds

- Some classes may have special limitations that a generic must fulfil
- Generic bounds allow you to set them
- Generic types can „extend" interfaces and classes
  - The actual type can then only be a sub-class of it

```
class Present<T extends Comparable<T>> {
    private T content;

    public boolean isBetterThan(Present<T> other) {
        return content.compareTo(other.content) > 0;
    }
}
```

Interface that defines the compareTo method

Using it here is safe

Recap: generic methods

- You can also have method-level generic types
  - Parameters and return type

```
public static <T> T printAndReturn(T t) {
    System.out.println(t);
    return t;
}
```

  - With bounding

```
public static <T extends Animal> T printAndReturn(T t) {
    System.out.println(t);
    return t;
}
```

  - Calling them

```
Seal seal = Printer.printAndReturn(new Seal("Count Seala“));
```

W07E04 Generic Container - Part 1

Not started yet.

Start exercise

Easy

Due tonight

10 min

1. Clone the exercise
2. Create a new class `Container`
   1. It should have one generic type
   2. It should have one attribute with that generic type to represent the content
   3. Add an empty constructor and a getter/setter for this attribute
3. Create a static method transfer with a generic type
   1. The method should transfer the content of one `Container` to another
   2. It takes `Container`s with the same generic type via 2 parameters
4. Test your implementation in your main class

W07E04 Generic Container - Part 2

Not started yet.

Start exercise

Easy

Due tonight

10 min

1. Add a bound to the class generic type
    1. Choose some interface from Java or create your own
    2. Then, adapt the transfer and main methods
2. **Challenge**: use a second generic type for your static method
    1. This second type should replace `Container<T>` for the return type and parameters
    2. Hint: You can already reference generic types while declaring generic types
3. Test your implementation in your main class

Outline

- Recap: Inheritance
- Recap: Abstract classes, interfaces & polymorphism
- Recap: Generics
- **Test supervised exercise**

Supervised exercise 1 information

- **Date:** Tuesday, 2 December 2025, 7:00 pm - 8:40 pm
- **Time**: 90 min + 10 min, **points**: 100
- **Content**: everything until the end of workshop week 07
- **Location**: Garching

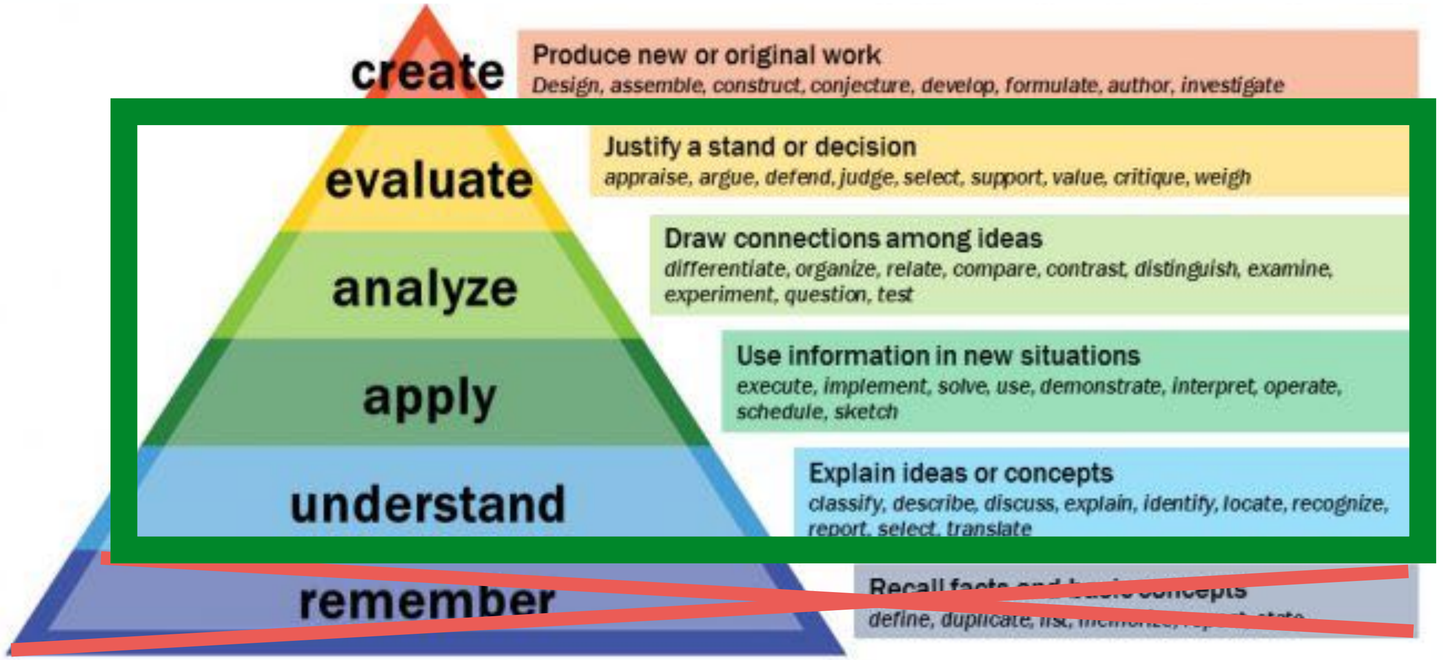
  You will receive an email with the actual lecture hall until Tuesday morning
- **Onsite**: you **must** participate in the assigned lecture hall
- **Setup**: use your own notebook
  - If you do **not** have a proper notebook, you can use a computer in a lab
    → send an email to itp.aet@xcit.tum.de asap (latest until Friday, November 28)
- **Rules**: read SE Policy: https://artemis.tum.de/courses/478/lectures/1751
- **Important: work alone, no communication is allowed!**

Artemis exam mode

- Exam mode features: https://artemis.tum.de/features/students
  - Guide: https://docs.artemis.cit.tum.de/user/exams/students_guide
  - Tutorials on https://artemis.tum.de/courses/478/lectures/1751
    1. Artemis Exam Mode
    2. Deactivate AI Features in your IDE
- **Test supervised exercise**
  - Try out the exam mode in Artemis and get familiar with it
  - You have 2 chances
    1. Live: this afternoon
    2. Asynchronous: from tonight until Sunday

General information

- The exercises focus on **understanding** and **problem solving**
- You **cannot** pass just with “learning by heart”
- Make sure that you are able to apply programming concepts to problem statements
- Review the learning goals of each lecture



Programming exercises

- Work on the programming exercise in your IDE
- Your code **must** compile on the build server
  - **Important**: compile failures will lead to 0 points
- No test feedback during the supervised exercise

Rules

- You must work on the supervised exercise **on your own**
  - **Important**: Do **not** use any kind of artificial intelligence (OpenAI, ChatGPT, GitHub Copilot, or any similar systems are forbidden, even if they are integrated in your IDE)
  - Do **not** use chat applications (keep them closed all time)
  - Do **not** post questions online
- You **must not** participate in the supervise exercise from home
- You may only use one monitor (no second monitor allowed)
- You must turn off all secondary devices (smartphone, tablet, etc.)
- More details in the SE Policy: https://artemis.tum.de/courses/478/lectures/1751
- Suspicious behavior, plagiarism and communication with other students is classified as cheating ("Unterschleif") and leads to consequences as mentioned in the APSO ("Allgemeine Prüfungs- und Studienordnung")
- In particular, the corresponding module in TUMonline will be marked as **failed (w. cheating)**

You can find instructions on Artemis

You must participate in the assigned lecture hall

🤖 Reminder: AI Usage

- AI Tools are no requirement to pass the course! **Use them responsibly!**

- 🔴 **Not allowed**: AI tools are **not** permitted during supervised exercises
- 🟡 **Use responsibly**: you may use GitHub Copilot* for programming tasks during exercises, homework, and the team project
  - **❗ *It must be disabled during supervised exercises; otherwise, it will be considered a cheating attempt**
- 🟢 **Recommended**: we encourage you to use **Iris** for all programming tasks during exercises, homework, and the team project on Artemis

Tips when using your own computer

- You are responsible for your computer: **before the supervised exercise**, make sure to install all required tools (browser, JDK 17, IntelliJ, git, etc.)
- Install all (operating system and application) updates before the exam: disable automatic updates for the duration of the exam
- **Close all windows and applications not needed for the supervised exercise**: this is especially important for all chat/communication applications
- Test your WiFi setup, and make sure you can connect to the different WiFi networks on campus: **eduroam** and **BayernWLAN**
- Keep the distractions to a minimum: disable notifications
- Check your git configuration for Artemis!
- Charge your battery
- Pack your laptop charger (and possibly an extension cord)
- Using a bluetooth mouse? Charge it

Technical issues

- If you experience technical issues, try to solve them on your own first (e.g. turn off WiFi and turn it on again, restart the computer, etc.)
- If you cannot solve the case on your own, raise your hand, a supervisor will try to help you
- In the unlikely case, the technical issue **cannot** be resolved (e.g. your computer breaks completely), you can resume the supervised exercise in the computer lab (“Rechnerhalle”)
  - In such a case, you will get additional time to compensate for the issue
  - Make sure that you are able to login on the computer lab in the FMI building

Next steps

- **Tutor group** exercises
  - T07E01 - A Seals magical fantasy world
  - T07E02 - Sealzar the Sorcerer
- **Homework** exercises
  - H07E01 - Superseal’s Clash of Warriors
  - H07E02 - Seal’s Magical List

Both exercises have optional parts that give additional bonus points

→ Due until **Wednesday, 3 December, 13:00**

Test supervised exercise

You have now the possibility to participate on Artemis

Duration: 90min
The workshop officially ends at 17:50

We will offer another chance to try out the exam mode on Artemis